
Sistemas Operacionais

Aula 03 - Interação entre Tarefas - Parte I

Rogério Pereira Junior

Instituto Federal de Santa Catarina
campus São José
rogeriopereirajunior@gmail.com

19 de maio de 2026



- 1 Comunicação entre tarefas
 - Atividades

- 2 Mecanismos de comunicação
 - PIPES
 - Filas de mensagens
 - Memória compartilhada

- 3 Finalizando



Comunicação entre tarefas



- **Cooperação:** Sistemas complexos são divididos em tarefas interdependentes que vão trabalhar em conjunto para atingir um objetivo comum.
- Por que construir sistemas com tarefas cooperantes?
 - **Atender usuários simultâneos:** servidor de banco de dados ou de e-mail.
 - **Computadores multicore:** acelerar a execução.
 - **Modularidade:** dividir sistema em módulos autônomos.
 - **Aplicações interativas:** navegadores Web e editores usam threads para melhorar interatividade

Para correta cooperação, a **comunicação** e a **coordenação** são necessárias!



Entendendo a necessidade de IPC

Imagine uma cozinha:

- **Tarefas:** O chef, o sub-chef e os garçons.
- **Comunicação:** O garçom precisa passar o pedido (dados) para o chef.
- **Coordenação:** O chef precisa avisar quando o prato está pronto para não esfriar.

Sem comunicação, o garçom não sabe o que entregar e o chef não sabe o que cozinhar.



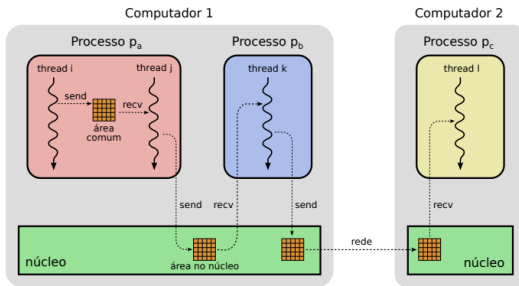
$$\mathbf{Cooperação} = \left\{ \begin{array}{c} \text{Comunicação} \\ + \\ \text{Coordenação} \end{array} \right.$$

- **Comunicação**: troca de informações entre tarefas
- **Coordenação**: ordem e dependência das ações das tarefas

Vamos discutir inicialmente a **Comunicação**



- A complexidade depende de onde as tarefas estão:
 - 1 **No mesmo processo (Threads):** Compartilham memória; a comunicação é via variáveis globais.
 - 2 **Processos distintos (Local):** Não há memória compartilhada; exige intervenção do Núcleo (Kernel) via chamadas de sistema.
 - 3 **Computadores distintos (Rede):** Exige protocolos de rede (Ex: TCP/IP).



- Genericamente se usa “IPC - Inter-Process Communication”



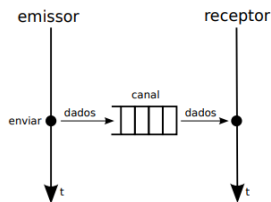
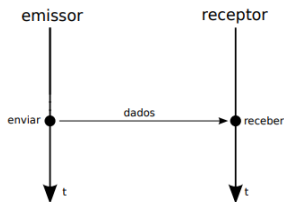
■ A comunicação pode ser:

- 1 Direta ou indireta
- 2 Síncrona ou assíncrona
- 3 Por mensagens ou por fluxos
- 4 Capacidade dos canais
- 5 Confiável ou não-confiável
- 6 Número de participantes



■ Comunicação direta:

- Emissor e receptor são identificados
- Enviar dados a uma tarefa: enviar (destino, dados)
- Receber dados de uma tarefa: receber (origem, dados)



■ Comunicação indireta:

- Emissor e receptor comunicam através de um canal
- Enviar dados ao canal: enviar (canal, dados)
- Receber dados do canal: receber (canal, dados)



■ Comunicação direta:

- O programador especifica o identificador (PID ou nome) do destinatário.
- **Exemplo Real:** Uma chamada telefônica para um ramal específico.
- **Vantagens:** Simplicidade de implementação
- **Desvantagens:** Acoplamento forte.

■ Comunicação indireta:

- O emissor coloca o dado em um canal compartilhado, e o receptor retira o dado desse mesmo canal
- enviar(caixa_postal_01, mensagem)
- **Analogia:** Um mural de avisos em um condomínio.
- **Vantagens:** Desacoplamento.
- **Exemplo Real:** O sistema de Logs do Windows ou o Syslog no Linux.

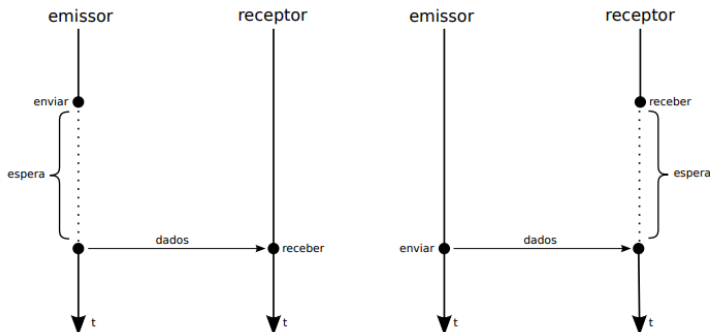


- Define como o tempo de execução do emissor e do receptor é afetado pela troca de dados.
- Basicamente, ele dita se uma tarefa precisa "parar e esperar" ou se pode continuar seu trabalho enquanto o dado viaja.



Sincronismo - Síncrona (ou bloqueante)

- As operações de envio/recepção **podem bloquear** as tarefas
- **Emissor Bloqueante:** O emissor envia o dado e fica "congelado"
- **Receptor Bloqueante:** O receptor tenta ler um dado e, se não houver nada disponível, ele para e fica esperando até que algo chegue.

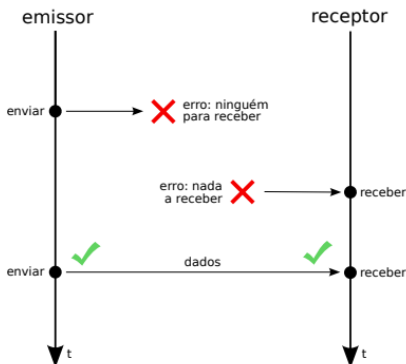


- **Analogia:** Uma chamada telefônica.



Sincronismo - Assíncrona (ou não-bloqueante)

- As tarefas não param para esperar.
- Emissor: Ele entrega o dado ao SO e volta a processar outras instruções.
- Receptor: Ele verifica se há mensagens.

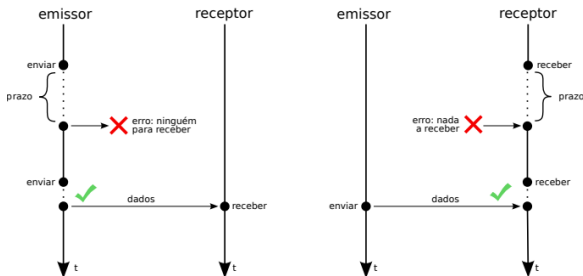


- Analogia: Enviar um E-mail ou WhatsApp.



Sincronismo - Semissíncrona (ou semibloqueante)

- É um meio-termo projetado para evitar o "travamento" eterno de um sistema.
- A tarefa se bloqueia para enviar ou receber, mas por um tempo máximo determinado (o timeout).



■ Resultado:

- Se a comunicação ocorrer dentro do prazo: Sucesso.
- Se o tempo acabar: A tarefa "acorda", recebe um erro de tempo esgotado e decide o que fazer (tentar novamente, avisar o usuário, etc.).

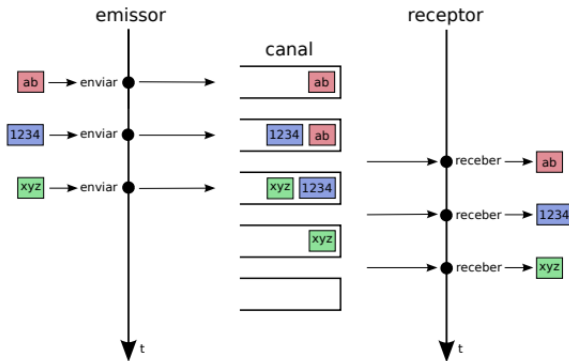


- semântica dos dados
- define como a informação é estruturada e entregue do emissor para o receptor.
- Sistema Operacional pode tratar os dados de duas formas:
 - como unidades discretas (mensagens)
 - como um fluxo contínuo (bytes).



Formato de envio: mensagens

- Mensagem: pacote de dados recebido pelo destino na íntegra.
- Os dados são enviados em "blocos" ou "pacotes" com início e fim bem definidos.

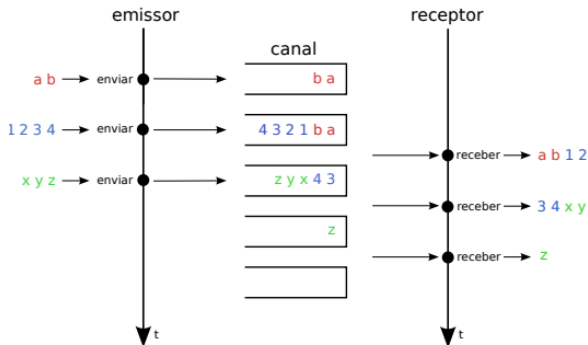


- Analogia: Cartas ou Telegramas.
 - Você não lê "metade de uma carta"



Formato de envio - fluxo de dados

- O canal de comunicação é visto como um arquivo.
- O emissor “escreve” dados no canal e o receptor os “lê” do canal



- Analogia: Uma mangueira de água ou Torneira.
 - A água flui continuamente.



- Se você precisa enviar comandos curtos e isolados (ex: "Ligar luz", "Abrir porta"), a Troca de Mensagens é melhor porque o receptor já recebe o comando pronto.
- Se você precisa transferir um volume enorme de dados (ex: um filme ou um backup), o Fluxo de Dados é mais eficiente, pois não gasta recursos processando cabeçalhos para cada pequeno pedaço de dado.



- Também chamada de buffering
- Define quanto espaço o Sistema Operacional reserva para armazenar mensagens que ainda não foram lidas pelo receptor.
- três situações devem ser analisadas
 - Capacidade nula ($n = 0$):
 - Capacidade infinita ($n = \infty$)
 - Capacidade finita ($0 < n < \infty$)



- O canal não tem memória nenhuma.
- A troca de dados só acontece quando o emissor e o receptor estão ambos prontos no exato mesmo momento.
 - É um encontro marcado.
- Sincronismo: O emissor sempre bloqueia até que o receptor execute a operação de leitura.
- Analogia: Passar o bastão em uma corrida de revezamento.
 - O corredor A não pode simplesmente jogar o bastão no chão e ir embora



- O emissor nunca precisa esperar.
 - Sempre pode enviar dados, que serão armazenados no buffer o canal enquanto o receptor não os consumir
- Realidade: Na prática, isso não existe em sistemas reais, pois a memória do computador (RAM) é finita.
- Útil no estudo dos algoritmos de comunicação e sincronização
- Analogia: O mar.
 - Você pode jogar quanta água quiser lá dentro que ele (teoricamente) nunca vai transbordar por causa da sua contribuição.

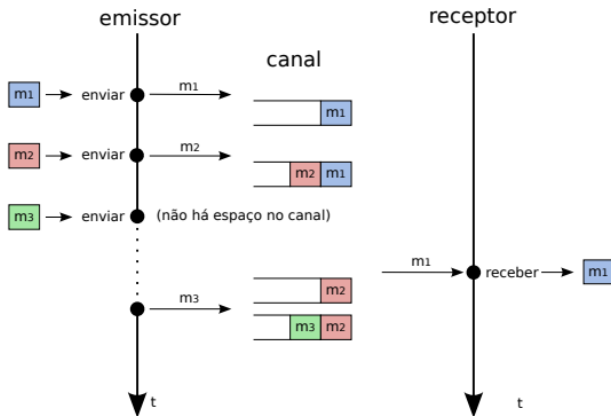


- O canal possui uma fila (buffer) com um tamanho n determinado
 - ex: 64 KB ou 10 mensagens
- Se o buffer estiver vazio, o receptor bloqueia (espera chegar algo).
- Se o buffer tiver espaço, o emissor deixa o dado e continua (assíncrono)
- Se o buffer estiver cheio, o emissor bloqueia (espera o receptor ler algo para abrir espaço)
- Analogia: Uma caixa de correio residencial.
 - Podemos colocar várias cartas lá dentro.
 - Se a caixa lotar de papel, o carteiro não consegue colocar mais nenhuma



Canal de capacidade finita

- Temos duas tarefas trocando dados através de um canal de comunicação com capacidade para duas mensagens e comportamento bloqueante.



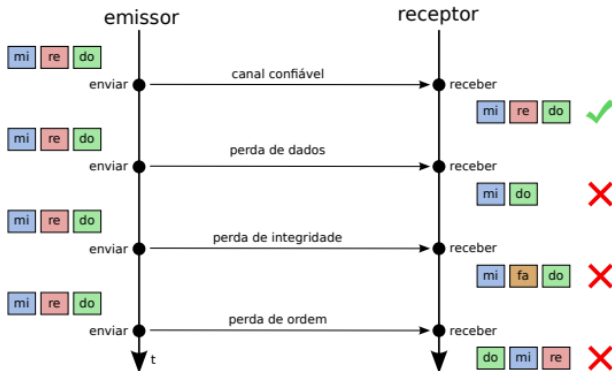
- Emissor bloqueado quando o canal estiver saturado



- Confiável: transporta ao destino todos os dados, mantendo a integridade e ordem de envio.
- Não-Confiável: podem ocorrer perdas:
 - De dados: dados enviados podem não ser recebidos
 - De integridade: dados podem chegar alterados
 - De ordem: os dados podem chegar fora de ordem



■ Canais no interior do SO: normalmente confiáveis



■ Via rede: protocolos básicos de comunicação são não-confiáveis

- protocolos de rede de nível mais elevado, como o TCP, permitem construir canais de comunicação confiáveis



- Existe situações onde uma tarefa necessita comunicar com várias outras
 - sistemas de chat ou mensagens instantâneas
- Define a topologia da interação
 - quantos processos podem estar "pendurados" nas extremidades de envio e de recepção



- 1:1 : um emissor e um receptor interagem através do canal de comunicação.
- modelo mais simples e comum, onde um canal liga estritamente dois processos.
- Analogia: Um tubo de PVC ligando duas salas.
 - Assopramos de um lado, o som sai apenas na outra ponta.
- Exemplo Real: Uma conversa privada entre duas pessoas



- M:N: quando um ou mais emissores enviam mensagens para um ou mais receptores.
- Frequentemente implementado através de um Barramento de Mensagens (Message Bus).
- Os processos "subscrevem" tópicos de interesse e publicam informações neles.
- Analogia: Uma Festa ou Conferência.
 - Várias pessoas falam e várias pessoas ouvem;
 - a comunicação é fluida e não depende de um par fixo de interlocutores.
- Duas situações distintas podem se apresentar neste caso:
 - Cada mensagem é recebida por apenas um receptor
 - Cada mensagem é recebida por todos os receptores



- Cada mensagem tem um só receptor.

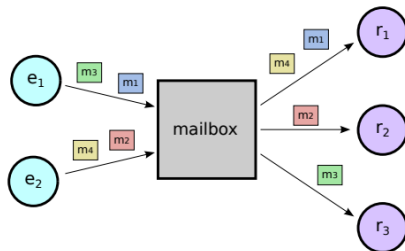


Figura: Comunicação M:N através de um mailbox.

- Implementada nas message queues do UNIX e Windows e nos sockets no protocolo UDP
- Na prática, o mailbox funciona como um buffer de dados



- Cada mensagem pode ter vários receptores.

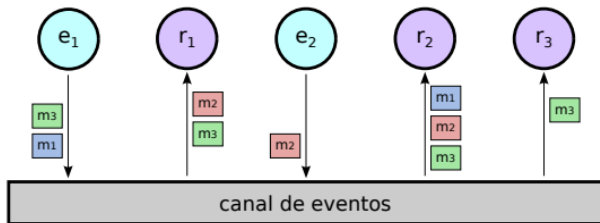


Figura: Comunicação M:N através de um barramento de mensagens

- Conhecida como
 - barramento de mensagens (message bus)
 - canal de eventos
 - canal publish-subscribe
- Em redes: multicast



Atividades



Disponível no sigaa



Mecanismos de comunicação



Comunicação entre Processos

Inter-Process Communication (IPC)

1. Pipes
(Fluxo de Bytes)

2. Filas
(Mensagens)

3. Memória
(Compartilhada)

- **Objetivo:** Como fazer processos isolados trabalharem em conjunto?



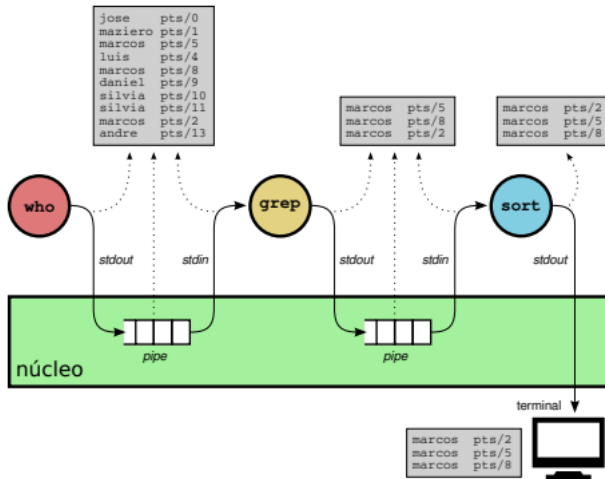
PIPES



- Forma mais simples de comunicação
- Fluxo unidirecional de dados entre dois processos relacionados.
- Pipe liga a saída de um processo à entrada de outro:
 - `who | grep marcos | sort`
- É como um cano de água: o que entra em uma extremidade sai na outra, obrigatoriamente na mesma ordem (FIFO - First In, First Out).
- OBS: Indireto, bloqueante, por fluxo, 1:1, confiável, capacidade finita



who | grep marcos | sort



Como o Pipe atua como IPC?

- Imagine que os processos são fábricas isoladas.
- Sem o IPC, elas não podem trocar mercadorias.
- O Pipe é a esteira rolante que conecta a saída de uma fábrica à entrada da outra.
- Fluxo: Unidirecional (Simplex). Os dados fluem apenas do "escritor" para o "leitor".
- Exemplo: `ls | grep "projeto"`
 - O processo `ls` produz dados.
 - O kernel do Linux gerencia o pipe para que esses dados sejam enviados ao processo `grep` sem precisar gravar um arquivo no disco.



- O SO cria um buffer (uma área de memória temporária).
- Quando o Processo A escreve no pipe, ele enche o buffer.
- Se o buffer lotar, o Processo A é bloqueado até que o Processo B leia algo e abra espaço.

Analogia da Linha de Produção:

- Pense em uma esteira rolante entre dois funcionários.
- O funcionário A coloca peças (dados) na esteira.
- Se o funcionário B não as pegar, a esteira para porque não há mais espaço.
- A ordem é rigorosa: a primeira peça colocada é a primeira a ser retirada.



Filas de mensagens



- Pipe: Esteira rolante
- Fila de Mensagens: caixa postal
 - Em vez de um fluxo contínuo de bytes (como no Pipe)
 - dados enviados em pacotes discretos chamados "mensagens".
- **Implementam o conceito de mailbox**
 - Envio e recepção ordenada de mensagens tipadas entre processos em um sistema operacional.
- Cada mensagem tem um tipo (número) e um conteúdo.

As filas de mensagens são mecanismos de comunicação entre vários processos (N:M ou N:1, dependendo da implementação), confiáveis, orientadas a mensagens e com capacidade finita.



- **Assíncrona:** O emissor pode colocar uma mensagem na fila e encerrar.
 - O receptor pode ler essa mensagem horas depois.
 - O kernel mantém a fila na memória.
- **Independência:** Diferente dos Pipes, não há necessidade de leitura e escrita simultânea para que o processo não bloqueie.
- **Prioridades:** o receptor pode dizer -> Eu quero ler a primeira mensagem do Tipo 10 (Urgente), mesmo que existam mil mensagens do Tipo 1 na frente.

Exemplo

Um sistema de log onde vários processos enviam mensagens de erro para uma fila centralizada, e um processo "monitor" as processa conforme sua capacidade.



Imagine uma empresa com uma caixa de correio central:

- **Vários Departamentos (Emissores):** Jogam envelopes na caixa.
- **Os Envelopes:** Têm cores diferentes. Vermelho para "Urgente", Azul para "Financeiro", Branco para "Geral".
- **O Gerente (Receptor):** Chega na caixa e, em vez de pegar o primeiro envelope do topo, ele procura especificamente pelos Vermelhos.

Se a caixa estiver cheia, quem for depositar um novo envelope tem que esperar alguém retirar um.

Se a caixa estiver vazia, o Gerente fica sentado ao lado dela esperando o carteiro chegar.



As filas de mensagens POSIX permitem a troca de mensagens estruturadas entre processos.

- **mq_open**: Cria uma nova fila ou abre uma existente.
- **mq_setattr** / **mq_getattr**: Gerenciam atributos como tamanho máximo e número de mensagens.
- **mq_send** / **mq_timedsend**: Envia mensagens. O emissor bloqueia se a fila estiver cheia (ou aguarda o *timeout*).
- **mq_receive** / **mq_timedreceive**: Recebe mensagens. O receptor bloqueia se a fila estiver vazia.
- **mq_close**: Fecha o descritor da fila no processo atual.
- **mq_unlink**: Remove a fila do sistema permanentemente.



Exemplo 1:

- O Emissor (sender.c)
 - Cria a fila e envia uma mensagem.
- O Receptor (receiver.c)
 - Abre a fila existente, lê a mensagem e depois remove a fila do sistema.
 - mq_attr: Se você tentar receber uma mensagem com um buffer menor do que o mq_msgsize definido na criação da fila, a função falhará.
- Compilação
 - gcc -Wall sender.c -o sender -lrt
 - gcc -Wall receiver.c -o receiver -lrt

Teste

- Rode o receiver sem rodar o sender
- Observe que teremos um erro, pois não tem nenhuma fila aberta.



- Arquivo `mqrecv.c`: recebe mensagens de uma fila de mensagens POSIX.
- Arquivo `mqsend.c`: envia mensagens para uma fila de mensagens POSIX.
- compilação
 - `gcc -Wall mqrecv.c -o mqrecv -lrt`
 - `gcc -Wall mqsend.c -o mqsend -lrt`

Resumo do Fluxo Combinado

- Terminal 1: Roda o `./mqrecv` -> Vai imprimir "Aguardando..." e fica parado.
- Terminal 2: Roda o `./mqsend` -> Vai imprimir "Sent message with value X".
- Resultado: Imediatamente, o Terminal 1 começará a mostrar "Received msg value X".



Analogia para entendimento

- Imagine que o **mqrecv.c** é um funcionário sentado na frente de um cesto (a fila) que cabe 10 bolinhas.
 - Ele define que o cesto só aceita bolinhas de gude (o tamanho do int).
 - Ele cruza os braços e fecha os olhos (mq_receive).
 - Assim que alguém joga uma bolinha, ela bate na mão dele, ele acorda, lê o número pintado na bolinha (printf), joga a bolinha fora e volta a dormir esperando a próxima.



Outro analogia!

- O `mqsnd` é o garçom que coloca pedidos na janelinha da cozinha.
- O `mqrcv` é o cozinheiro que retira.
- Se o cozinheiro sair para almoçar e o garçom continuar colocando pedidos, vai chegar uma hora que a janelinha fica cheia e o garçom tem que segurar o pedido na mão até que surja um espaço.



Sistema de "Chat de Prioridade-- **chat.c**

Imagine que temos um sistema onde usuários enviam mensagens comuns, mas o administrador pode enviar um comando de "SHUTDOWN" que deve ser processado antes de qualquer outra mensagem.



Memória compartilhada



Criar uma área de memória onde processos leem e escrevem

- IPC mais rápido que existe no Linux.
- Pipes e as Filas de Mensagens exigem que o kernel copie os dados do espaço de um processo para um buffer e depois para o segundo processo
 - Memória compartilhada elimina esse "intermediário".
 - Comunicação rápida, sem interferência do núcleo
- Adequada para muitos dados compartilhados
- Ausência de mecanismos de coordenação



- É o mais rápido porque não há cópia de dados entre o processo e o Kernel.
- O dado é escrito direto na RAM.
 - A comunicação ocorre na velocidade da própria memória RAM
 - Ideal para aplicações que precisam processar volumes gigantescos de dados em tempo real
- Problema: Condição de Corrida
 - Se dois processos escreverem no mesmo bit ao mesmo tempo, o dado corrompe.

Analogia com Google Doc

- Imagine um documento compartilhado
- Várias pessoas podem editar ao mesmo tempo.
- Se não houver uma coordenação (como o cursor mostrando onde cada um está)
- Uma pessoa pode apagar o que a outra acabou de escrever



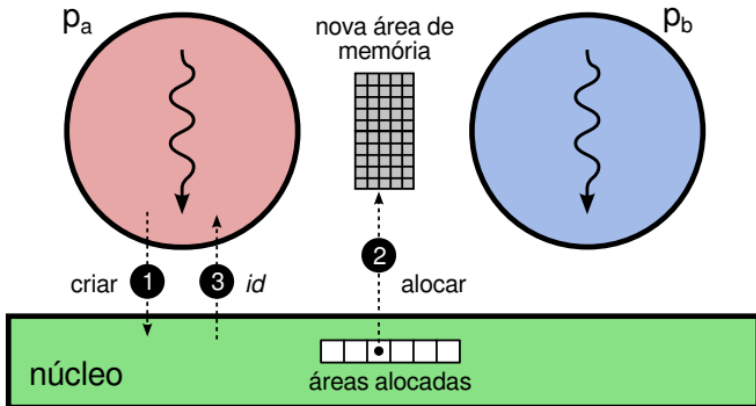
O SGBD (como Oracle ou PostgreSQL) mantém uma grande área de memória compartilhada (SGA/Buffer Cache) onde os dados das tabelas ficam carregados.

Diversos processos de usuários leem e escrevem ali simultaneamente para evitar acessos lentos ao disco rígido.



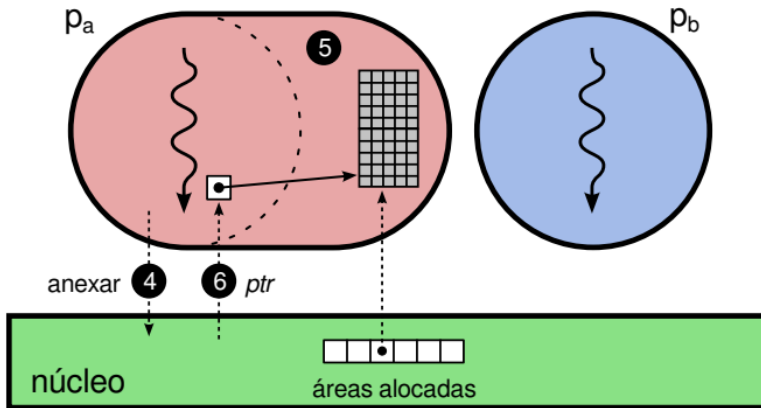
Memória compartilhada - Etapas

- 1 p_a solicita ao núcleo uma área de memória compartilhada
- 2 o núcleo aloca uma nova área de memória
- 3 o núcleo devolve ao processo p_a o id dessa área



Memória compartilhada - Etapas

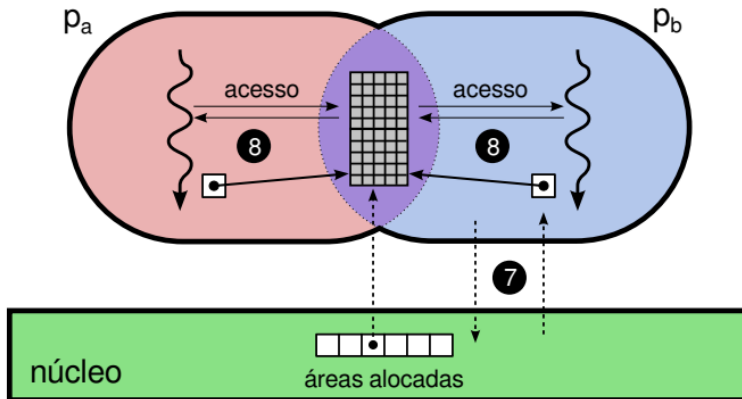
- 4 p_a pede ao núcleo para anexar a área id à sua memória
- 5 o núcleo ajusta os mapas de memória de p_a
- 6 o núcleo devolve a p_a um ponteiro para a área id



Memória compartilhada - Etapas

7 p_b executa os passos 4-6 e também recebe um ponteiro para a área id

8 Os processos p_a e p_b acessam a área compartilhada id



- Imagine que cada processo vive em sua própria "bolha" de memória RAM
 - um processo não consegue ver o que está dentro da bolha do outro.
- A Memória Compartilhada é como se o sistema operacional mapeasse o mesmo bloco físico de RAM para dentro da bolha de ambos os processos.
- Fluxo:
 - Criação: Um processo solicita ao sistema operacional a criação de um segmento de memória compartilhada.
 - Mapeamento (Attach): Outros processos "anexam" esse mesmo segmento ao seu próprio espaço de endereçamento.
 - Acesso Direto: A partir daí, qualquer dado escrito por um processo está instantaneamente disponível para os outros, sem chamadas de sistema (system calls) adicionais.



O Cenário: O Escritor e o Leitor

Dividiremos a comunicação em dois programas independentes que acessam o mesmo objeto de memória: `/meu_quadro`.

1. escritor.c

- Cria o objeto de memória no Kernel.
- Define o tamanho (espaço para 1 int).
- Mapeia o espaço para um ponteiro.
- **Ação:** Escreve o valor **42** e encerra.

2. leitor.c

- Abre o objeto já existente.
- Mapeia o espaço para seu próprio ponteiro.
- **Ação:** Lê o valor, imprime na tela e remove a memória do sistema (*unlink*).

O que observar?

- **Persistência:** O Escritor pode terminar antes do Leitor começar. O dado "sobrevive" no Kernel.
- **Não-bloqueio:** Se o Leitor rodar primeiro, ele não espera; ele lerá o que estiver lá (geralmente lixo ou zero).



O Cenário: Escrita e Leitura Dinâmica

Este exemplo demonstra como transformar um "objeto de sistema" em uma variável utilizável via ponteiros, operando em um ciclo contínuo.

■ A Criação do Terreno:

- O sistema reserva um nome no Kernel (`/sharedmem`).
- O espaço é "esticado" (`ftruncate`) para o tamanho exato do dado.

■ O Elo de Ligação (Mapeamento):

- A função `mmap` cria uma ponte entre a memória do sistema e o programa.
- Alterações no ponteiro `*ptr` refletem instantaneamente na RAM física.

■ O Fluxo em Tempo Real:

- O processo gera valores aleatórios e os "deposita" na memória.
- Ele mesmo (ou qualquer outro processo conectado) pode resgatar esse valor a qualquer momento.

OBS: MAP_SHARED

Diferente de uma cópia comum, o sinalizador `MAP_SHARED` garante que a memória seja um "recurso público": o que você escreve, o mundo (processos autorizados) vê.



Cenário de Monitoramento Industrial

Simulação de um sistema onde um **Processo Sensor** alimenta um **Painel de Controle (Monitor)** através de uma ficha técnica compartilhada.

■ A "Ficha" de Dados (Struct):

- Não compartilhamos apenas um número, mas um conjunto: **Temperatura + Pressão**.
- A memória compartilhada é moldada para o tamanho exato da struct.

■ Produtor vs. Consumidor em Velocidades Diferentes:

- **Sensor:** Atualiza os dados a cada 1 segundo.
- **Monitor:** Lê e exibe os dados a cada 2 segundos.
- *Resultado:* A memória atua como um "estado atual" sempre disponível, independente da frequência de leitura.

■ Persistência do Estado:

- Se o Sensor for desligado, o Monitor continuará lendo o **último valor registrado**.
- O dado não "flui" como em um pipe; ele "reside" na RAM.

OBS

A memória compartilhada sobrevive ao fim do processo escritor. O dado só deixará de existir quando o Sistema Operacional for reiniciado ou quando um processo executar o comando `shm_unlink`.



Finalizando



- Os processos são isolados pelo Sistema Operacional para segurança.
- **IPC (Inter-Process Communication):** Conjunto de mecanismos que permitem que processos troquem dados.
- **Principais mecanismos:**
 - 1 Pipes (Tubulações)
 - 2 Filas de Mensagens (Message Queues)
 - 3 Memória Compartilhada (Shared Memory)



Conceito

Funciona como um duto unidirecional de bytes entre dois processos.

- **Modelo:** FIFO (*First In, First Out*).
- **Analogia:** Um walkie-talkie ou um tubo pneumático.
- **Exemplo Real:** `ls | grep "pdf"` no terminal Linux.
- **Sincronização:** O SO gerencia o bloqueio se o "tubo" estiver cheio ou vazio.



- **Comunicação:** **Indireta**. Os processos não escrevem um no outro, mas sim em um canal (o pipe) gerenciado pelo Kernel.
- **Síncrona ou Assíncrona:** **Síncrona**. Se o pipe estiver vazio, o leitor bloqueia; se estiver cheio, o escritor bloqueia.
- **Mensagens ou Fluxos:** **Por Fluxos (Bytes)**. O dado entra como uma corrente contínua; o receptor não sabe onde começa ou termina uma "mensagem" a menos que ele mesmo trate isso.
- **Capacidade dos Canais:** **Limitada**. O Kernel reserva um buffer pequeno (geralmente 64KB no Linux).
- **Confiável:** **Confiável**. O Kernel garante que os bytes cheguem na mesma ordem em que foram enviados.
- **Número de Participantes:** **Exatamente 2**. Geralmente um pai e um filho (no caso de pipes anônimos).



Conceito

Troca de blocos estruturados de dados (mensagens) via Kernel.

- **Vantagem:** Permite atribuir **prioridades** às mensagens.
- **Analogia:** Uma caixa de correio ou WhatsApp.
- **Persistência:** A mensagem fica no Kernel até que alguém a leia.
- **Chamadas POSIX:** `mq_open`, `mq_send`, `mq_receive`.



Funciona como um sistema de correio organizado, onde os dados têm "identidade".

- **Comunicação:** **Indireta**. Os processos depositam e retiram mensagens de uma fila identificada por uma chave ou nome no Kernel.
- **Síncrona ou Assíncrona:** **Assíncrona**. O emissor pode colocar a mensagem na fila e continuar trabalhando (ou até encerrar), mesmo que o receptor ainda não esteja pronto.
- **Mensagens ou Fluxos:** **Por Mensagens**. Os dados são pacotes discretos com tamanho e tipo definidos.
- **Capacidade dos Canais:** **Limitada**. Definida pelo administrador do sistema (ex: máximo de 10 mensagens na fila).
- **Confiável:** **Confiável**. As mensagens permanecem no Kernel até serem lidas e respeitam a ordem de prioridade.
- **Número de Participantes:** **Múltiplos**. Vários processos podem escrever e vários podem ler da mesma fila.



Conceito

Mapeia um pedaço da RAM física para o espaço de vários processos.

- **Velocidade:** É o mecanismo mais rápido (acesso direto à RAM).
- **Analogia:** Um quadro negro em uma sala comum.
- **Risco:** Requer sincronização manual para evitar corrupção de dados.
- **Chamadas POSIX:** `shm_open`, `ftruncate`, `mmap`.



É o mecanismo de "alto desempenho", onde os processos dividem o mesmo espaço físico.

- **Comunicação:** **Direta**. Uma vez estabelecido o mapeamento, os processos acessam a mesma memória RAM física sem intermediação constante do Kernel.
- **Síncrona ou Assíncrona:** **Totalmente Assíncrona**. O sistema não oferece bloqueio nativo. Se um escreve, o outro não "acorda" automaticamente; ele precisa checar a memória.
- **Mensagens ou Fluxos:** **Nenhum dos dois (Acesso Direto)**. Os dados são tratados como variáveis, estruturas (structs) ou vetores comuns na memória.
- **Capacidade dos Canais:** **Alta**. Limitada apenas pela memória RAM disponível e pelo tamanho definido no ftruncate.
- **Confiável:** Não há garantia nativa contra corrupção se dois processos escreverem ao mesmo tempo. Exige o uso de mecanismos de sincronização.
- **Número de Participantes:** **Múltiplos**. Qualquer processo que tenha permissão e conheça o nome da área pode mapeá-la.



Característica	Pipes	Filas	Mem. Comp.
Dados	Bytes	Mensagens	Estruturas
Velocidade	Média	Média	Altíssima
Sincronização	Automática	Automática	Manual
Persistência	Temporária	No Kernel	No Kernel

Escolha o Pipe pela simplicidade, a Fila pela organização e a Memória Compartilhada pela performance.



Tabela: Comparativo: Fila de Mensagens vs. Memória Compartilhada

Característica	Fila de Mensagens (Correio)	Memória Compartilhada (Quadro)
Persistência	A mensagem desaparece da fila após ser lida por um processo.	O dado permanece na memória até que seja explicitamente sobrescrito ou removido.
Sincronização	Nativa: O SO bloqueia o receptor e avisa quando novos dados chegam.	Manual: O processo deve chamar mudanças ou usar Semáforos/Mutexes.
Cópia de Dados	O SO realiza cópias entre o espaço do usuário e o Kernel (mais lento).	Sem cópias intermediárias: acesso direto à RAM física (maior velocidade).



Guichê de Farmácia

- Pipes: É um cano onde você joga moedas. Elas saem do outro lado exatamente na ordem. Se o cano entupir, você para de jogar.
- Memória Compartilhada: É um caderno em cima do balcão. Eu escrevo, você apaga, outro lê. Se não houver educação, vira bagunça.
- Fila de Mensagens: É a caixa de receitas.
 - O cliente (Emissor) coloca a receita lá e vai embora.
 - A receita tem um grau de urgência (Prioridade).
 - O farmacêutico (Receptor) chega, olha as receitas na caixa e escolhe a mais urgente para processar. Se a caixa estiver vazia, ele senta e espera até alguém colocar uma receita.



■ Fila de Mensagens (Message Queues)

- Sistemas de Tempo Real (priorização de alarmes críticos).
- Arquiteturas de Microserviços (comunicação assíncrona).
- Servidores de impressão (gerenciamento de jobs).

■ Pipes (Dutos)

- Shell do Linux: `cat arquivo | grep "erro"`.
- Redirecionamento de logs de um processo para outro.
- Comunicação simples entre Processo Pai e Filho.

■ Memória Compartilhada (Shared Memory)

- Banco de Dados (SGA no Oracle, buffers de alto desempenho).
- Processamento de vídeo/imagem em tempo real (frames pesados).
- Jogos de alta performance (troca de estados entre motor e IA).

Pesquise o porque de usar fila de mensagens e não memória compartilhada em sistemas de tempo real e microserviços

